



流的各种创建方式

北京理工大学计算机学院
金旭亮

概述



为了支持级联调用，所有流的“中间操作”都会返回一个流对象。



但我们总需有一个“初始”的流对象，以便基于它开始构建一个流处理管线。



下面的PPT介绍如何从原始数据源中创建这个“初始”的流对象。

基于集合创建Stream

所有集合对象都实现的Collection接口定义了一个Stream()或parallelStream()方法，可以通过它来创建流对象：

CreateStream.java

```
// 基于集合对象创建流
private static void createForCollection() {
    //创建一个集合对象
    Set<String> names = new HashSet<>();
    names.add("Tom");
    names.add("Jacket");
    // 基于集合对象创建流
    Stream<String> sequentialStream = names.stream();
    //基于集合对象创建并行流 ( parallel stream)
    Stream<String> parallelStream = names.parallelStream();
    //基于数组创建流
    int[] numbers = {2, 3, 5, 7, 11, 13};
    var arrayStream : IntStream = Arrays.stream(numbers);
}
```

JDK中的Arrays类提供了直接基于现有数组创建流的静态方法stream()，注意一下流的类型比较特殊，是IntStream。

JDK中流的分类



顺序执行的流
(Stream)

并行执行的流
(ParallelStream)

对象流
(Stream<T>)

原始数值流
(IntStream等)

使用Stream.of创建流

Stream接口定义了一个of方法，可以用于创建流……

```
//使用Stream.of方法创建流
private static void createUseStreamOf(){
    //使用可变参数创建流
    var strStream : Stream<String> = Stream.of("Java", "Kotlin", "C#");
    //基于数组创建流
    String[] names = {"Ken", "Jeff", "Chris", "Ellen"};
    Stream<String> nameStream = Stream.of(names);
}
```



在Java标准库中，of方法的实现实际上被委托给java.util.Arrays类定义的stream方法，基于数组创建流。

Stream.Builder<T>

使用Stream.Builder<T>也能创建流.....

```
//使用StreamBuilder创建流
private static void createUseStreamBuilder(){
    Stream<String> stream = Stream.<String>builder()
        .add("Ken")
        .add("Jeff")
        .add("Chris")
        .add("Ellen")
        .build();
}
```

基于数组构建流

JDK中的Arrays类提供了直接从对象数组中创建流的静态方法stream(), 以字符串数组为例:

```
// 基于字符串数组创建一个字符串流  
Stream<String> names = Arrays.stream(  
    new String[] {"Ken", "Jeff"});
```



Arrays.stream()方法提供了用于处理int[]、double[]与long[]型数组的重载形式, 生成相应的原始数值流。

构建一个不包含任何元素的流

在有些情况下，需要构建一个“空”的流，调用`Stream.empty()`方法可以达到这个目的：

```
//构建一个不包含任何元素的流
private static void createEmptyStream() {
    //构建一个空的字符串流
    Stream<String> stream = Stream.empty();
    //构建一个空的整数流
    IntStream numbers = IntStream.empty();
}
```


“基于工厂函数”构建无限流

我们可以定义一个函数，这个函数可以帮助我们构建出流的元素，这个函数可以看成是生成流元素的“元素工厂”，然后使用这个工厂生产出来的“元素”构建出流。

```
//使用流元素工厂构建流
private static void generateExample() {
    //整数生产“流水线”，每次调用，生成一个[0,100)区间内的整数
    Supplier<Integer> intFactory = () -> (int) (Math.random() * 100);
    //调用整数元素“工厂”生产出10个“整数” 产品
    Stream<Integer> stream = Stream.generate(intFactory).limit(10);
    //输出所有元素
    stream.forEach(System.out::println);
}
```

示例：StreamElementFactory.java



注意：这种方式可以构建出元素数目无限的“无限流”，因此，通常都会使用**limit()**方法限定要生成的元素个数。

“基于迭代函数”构建无限流

有些流的元素需要依据它的前一个元素才能确定，对于这种情况，可以使用`Stream.iterate()`方法构建流对象：

```
//使用迭代法构建流
private static void iterateExample() {
    //起始值为1
    int seed = 1;
    //使用递推公式:  $a[n]=a[n-1]+2$  生成奇数
    UnaryOperator<Integer> intFactory = n -> n + 2;
    //生成包容10个奇数的流
    Stream<Integer> stream = Stream.iterate(seed, intFactory).limit(10);
    //输出所有元素
    stream.forEach(System.out::println);
}
```

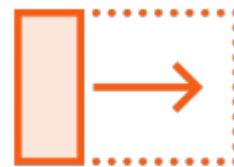
动手动脑



- 想一想，如何生成一个常量流？比如生成一个流，其中的元素值全部都为100。
- 请动手一试！

- Fabonacci数列是一个非常著名的数列，每个元素，都是它前两个元素之和，请使用Stream API生成这一数列的前10个元素。
- 参考解答：CreateFibonacci.java

无穷（或无限）流



使用`Stream.generate`和`Stream.iterate`方法构建的流，可以拥有“无数个元素”，称为“**无穷流 (infinite stream)**”。

当然我们不能直接使用这个流，通常会使用`limit()`方法构建一个仅包容“有限元素”的流，否则，程序运行起来就“没完没了”了。



如果使用RxJava等第三方库，我们还可以完成更复杂与精细的操作，比如，每隔一秒钟“发射 (emit)”一个数给使用者。有关这块内容，会在专门的课程中介绍。

应用实例：生成“随机数”流

使用Stream.generate()方法，配合JDK中所提供的Math.random()方法，可以很方便地生成特定区间内的随机整数。

```
//使用generate()方法生成[from,to]之间的随机整数
private static void useGenerate(int from, int to) {
    Stream.generate(Math::random)
        .limit(5)
        .map(doubleNum -> (int) (doubleNum * (to - from + 1)) + 1)
        .forEach(System.out::println);
}
```

生成随机数值流的简便方法

鉴于程序中经常需要生成随机的整数和浮点数，JDK8中为Random类添加了ints()/longs()/doubles()方法创建相应的数值流。

```
//基于JDK所提供的Random类方法生成随机数流
private static void useRandomClass() {
    //生成[0,100)区间中的随机整数
    new Random()
        .ints( randomNumberOrigin: 0, randomNumberBound: 100)
        .limit(5)
        .forEach(System.out::println);

    //生成[100,200)区间内的随机浮点数
    new Random()
        .doubles( randomNumberOrigin: 100, randomNumberBound: 200)
        .limit(5)
        .forEach(System.out::println);
}
```

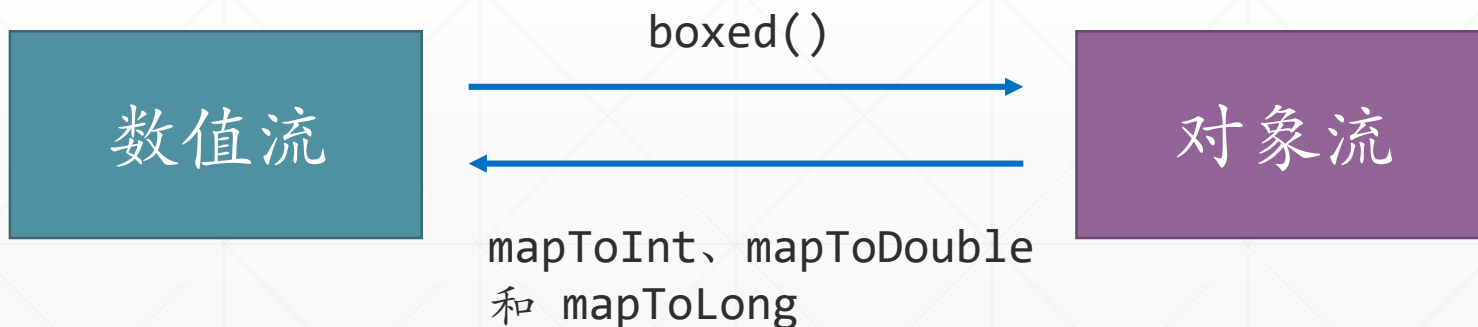
数值流



引入数值流的主要考虑，是避免装箱损失。



Java 8 引入了三个原始类型流接口：`IntStream`、`DoubleStream` 和 `LongStream`，其流中的元素为 `int`、`long` 和 `double` 这样的原始数据类型，在处理数据时，可以避免装箱带来的性能损失。



使用装箱流

有些常用的流操作，无法用于数值流，这时，可以使用boxed方法或mapToObj方法将数值流转换对象流，就可以使用这些操作了。

```
public class UseBoxedStream {  
    public static void main(String[] args) {  
        //以下代码无法编译  
        //IntStream.of(3, 1, 4, 1, 5, 9)  
        //        .collect(Collectors.toList());  
  
        //解决之道：将int用boxed方法转换为Integer，就可以编译了  
        List<Integer> ints = IntStream.of(3, 1, 4, 1, 5, 9)  
            .boxed()  
            .collect(Collectors.toList());  
        //输出: [3, 1, 4, 1, 5, 9]  
        System.out.println(ints);  
  
        //另一种方式：使用mapToObj，将其转换为对象  
        List<Integer> ints2 = IntStream.of(3, 1, 4, 1, 5, 9)  
            .mapToObj(Integer::valueOf)  
            .collect(Collectors.toList());  
        //输出: [3, 1, 4, 1, 5, 9]  
        System.out.println(ints2);  
    }  
}
```

基于文件和文件夹创建流-1

Java 8中，我们可以基于一个文本文件中的所有行构建流，也可以基于一个文件夹中的所有子文件夹构建流。

```
private static void useNIOFileStream() {  
    //lines()方法，返回的就是一个字符串流  
    try (Stream<String> lines = Files.lines(  
        Paths.get("data.txt"))) {  
        lines.forEach(System.out::println);  
    } catch (IOException ex) {  
        System.out.println(ex.getMessage());  
    }  
}
```

参看示例：GetStreamFromJDK.java

java.nio.file.Files 中的很多静态方法都会返回一个流。

```
public static Stream<String> lines(Path path, Charset cs) throws IOException
```

基于文件和文件夹创建流-2

```
// 列出当前文件夹下的所有文件
public static void listFileTree() {
    Path dir = Paths.get( first: "");
    System.out.printf(" %s contains: \n", dir.toAbsolutePath());
    //walk方法返回一个Stream
    try (Stream<Path> fileTree = Files.walk(dir)) {
        fileTree.forEach(System.out::println);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```



Files和Paths都是Java NIO中的类，Java IO和NIO是一个比较大的话题，本网站提供有专门的技术专题课程进行介绍。

动手动脑



- 创建一个生成素数的流，然后使用它找出[100,10000]内的素数
- 先使用标准流执行它，再使用并行流执行它，比对这两种方式的执行效果

想得到一个生成素数的流，可以参考示例PrimeUtil.java

动手动脑



请编写一个程序，使用`lines()`方法读入指定文件夹下所有`*.java`文件的内容，统计其总行数，并且按照以下标准将文件进行分组：

- (1) 小文件：代码总行数少于50。
- (2) 中文件：代码总行数在`[50,100)`之间
- (3) 长文件：代码总行数大于100

统计行数时，空行不应该计算进去。